

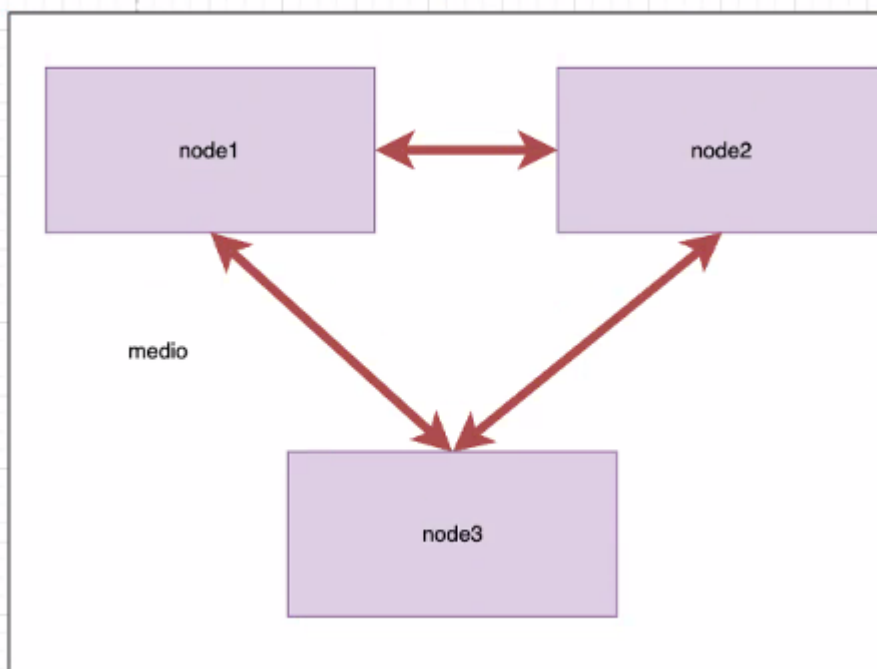
Apunte 5

Priscilla Romero Barquero - 2023332718

Sistemas distribuidos

Normalmente, cuando se monta una red sin importar la segregación que tenga hay que prestar atención a varias cosas. En un sistema distribuido se van a tener varios nodos, generalmente un mínimo de 3, aunque pueden ser más.

Cuando se tienen varios nodos, en algún momento van a tener que sincronizarse. Esa sincronización tiene un impacto directo en cómo se comunican entre ellos y en cómo manejan esa sincronización, pero siempre ocurre dentro de una red.



Siempre se protege el nodo por sí solo, pero si se sobrepasa un límite, los datos van a salir del nodo y van a viajar por algún medio, ya sea por conexión wired o wireless.

- En el caso de wired, las redes modernas trabajan con conexiones punto a punto. Esto significa que cuando los nodos se comunican, establecen una conexión directa entre los servidores, lo que hace más difícil que alguien pueda escuchar el tráfico de la red. Para lograrlo, alguien tendría que meterse en un circuito virtual dentro del sistema de red, lo cual no es sencillo.
- En el caso de wireless, la comunicación se maneja por difusión. Cuando se hace una transferencia de datos en un medio inalámbrico, normalmente los datos salen en todas las direcciones.

Ataques comunes

Man in the middle

El ataque man in the middle ocurría con mucha frecuencia hace años, cuando las redes utilizaban un hub para la comunicación. Este dispositivo trabajaba por difusión, así que al conectar varios servidores a sus puertos, la información se enviaba a todos los puertos, aunque solo hubiera dos dispositivos realmente comunicándose. Debido a este comportamiento, era posible que alguien interceptara y robara la información que se estaba transmitiendo, y de ahí viene el nombre del ataque.

Hub:



En las redes modernas se utiliza un switch. Internamente, el switch crea un camino específico por donde pasan los datos entre los dispositivos que se están comunicando. Debido a esto, la única forma de obtener esos datos es lograr comprometer o hackear el switch directamente.

Switch:



Entonces, en redes wired no es tan común que este tipo de ataques ocurra, pero ese ataque nació justamente por las herramientas que se usaban en ese momento.

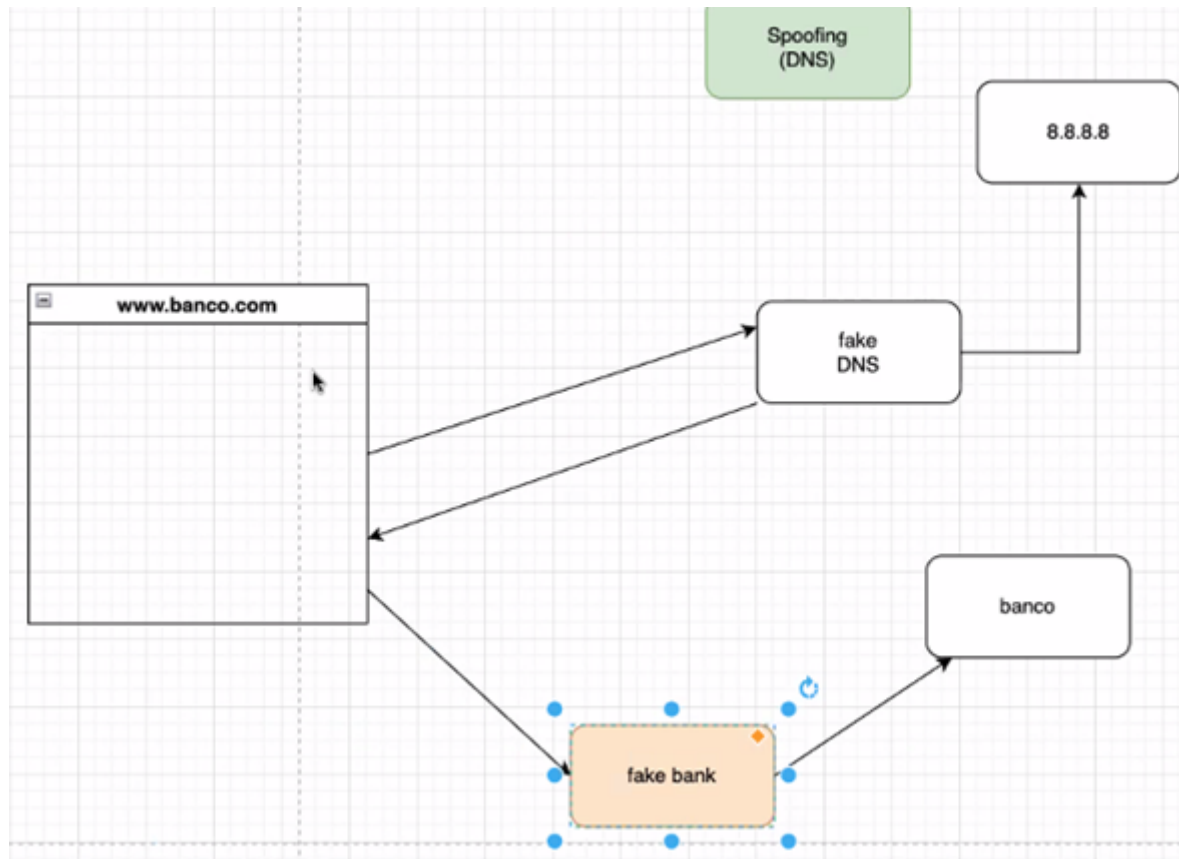
Cuando aparece el cloud, la comunicación empieza a pasar por internet o por una red pública, es decir, de un nodo interno hacia uno externo. Esto implica la posibilidad de que alguien pueda realizar un ataque durante el tránsito de los datos. Por esa razón siempre se debe implementar TLS (Transport Layer Security) para evitar que alguien pueda intervenir la comunicación.

Algo importante a tener en cuenta en la encriptación de red es que al aumentar la cantidad de certificados y procesos de cifrado, también aumenta el delay al hacer consultas. Esto se debe a que la petición y la respuesta deben encriptarse y desencriptarse, lo cual suma tiempo. Entre más capas se agreguen, más aumenta ese tiempo, porque está asociado al overhead.

Spoofing DNS

Cuando se utiliza un servidor DNS, su función es resolver nombres y devolver la dirección IP correcta. Por ejemplo, si alguien quiere entrar a la página de un banco, el DNS legítimo devuelve la IP real del sitio del banco y la persona accede normalmente. En un ataque de DNS spoofing, lo que ocurre es que el atacante coloca un DNS falso. La computadora de la víctima termina usando ese DNS fake en lugar del DNS real. Debido a esto, la computadora cree que está consultando al servidor legítimo. Entonces, si la persona intenta

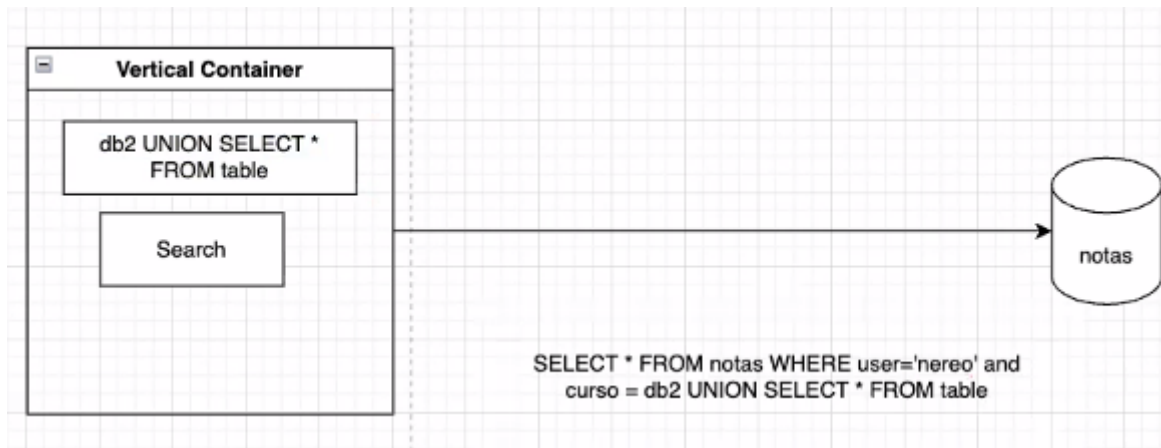
entrar al banco, la petición no llega al servidor público verdadero, sino al DNS falso. Este DNS fake redirige primero a un sitio falso que imita al banco, pero además puede comunicarse con el banco real para mantener la apariencia de que todo funciona correctamente. El resultado es que la persona usuaria piensa que está interactuando con el banco real, cuando en realidad está siendo interceptada.



Esto mismo puede ocurrir con las bases de datos. Cuando se hace resolución de nombres de la base de datos utilizando DNS, alguien puede interceptar esas resoluciones y redirigirlas hacia una base de datos falsa. Esa base de datos falsa puede estar diseñada para robar información mientras al mismo tiempo envía los datos a la base de datos real para que todo parezca normal. De esta forma, tanto la persona como la aplicación creen que están comunicándose con la base de datos legítima, cuando en realidad están pasando por una base de datos falsa que está interceptando la información.

SQL Injection

Supongamos que se le permite al usuario realizar una búsqueda. En el momento en que se construye la consulta, ¿qué sucede? Si la consulta se arma directamente con lo que el usuario escribe, esa persona podría insertar cualquier cosa en ese campo. Al hacerlo, está inyectando un statement SQL dentro de la consulta original. Esto significa que, cuando se arma la consulta completa, se combina la parte que debería venir del sistema con lo que escribió el atacante y la consulta final puede ejecutar instrucciones no autorizadas.

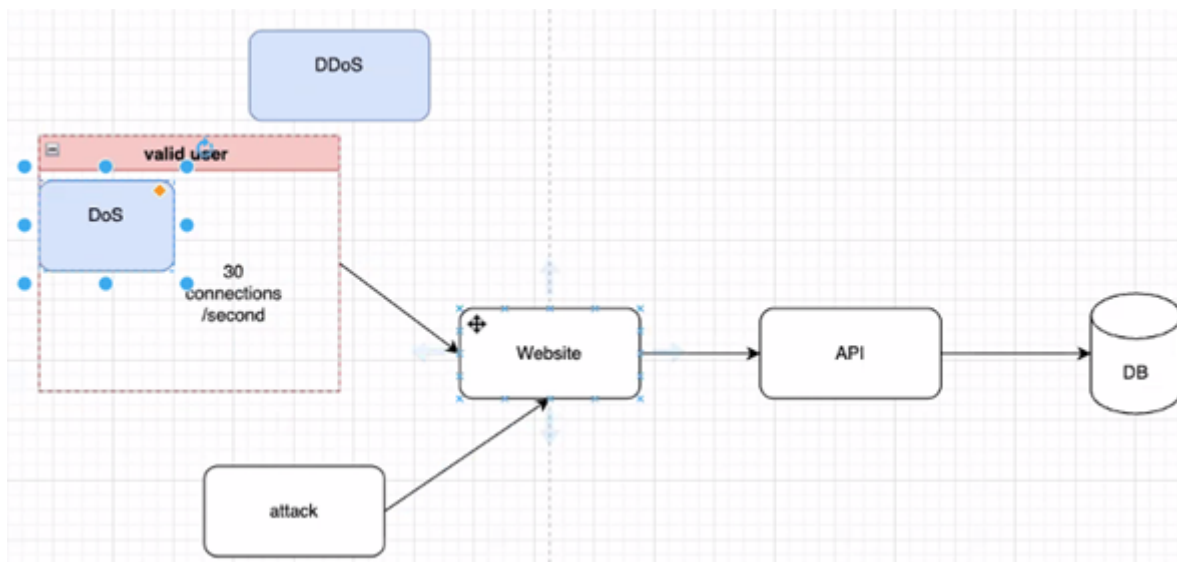


DDoS

Se tiene un sistema común con una infraestructura física que tiene limitaciones: memoria, CPU y una cantidad máxima de peticiones recurrentes que puede atender. Un ataque DDoS apunta justamente a alguno de esos puntos débiles.

Por ejemplo, si se ataca un sitio web con miles de peticiones al mismo tiempo, el sistema empieza a crear threads y realizar muchos context switches. Esto consume memoria y recursos muy rápido. En algún momento, cuando un usuario real intenta entrar al sitio, el sistema está tan cargado que debe hacer demasiado scheduling y el tiempo de respuesta aumenta. Esto provoca que el usuario reciba un timeout o que el sistema simplemente se caiga, haciendo que el usuario abandone el servicio.

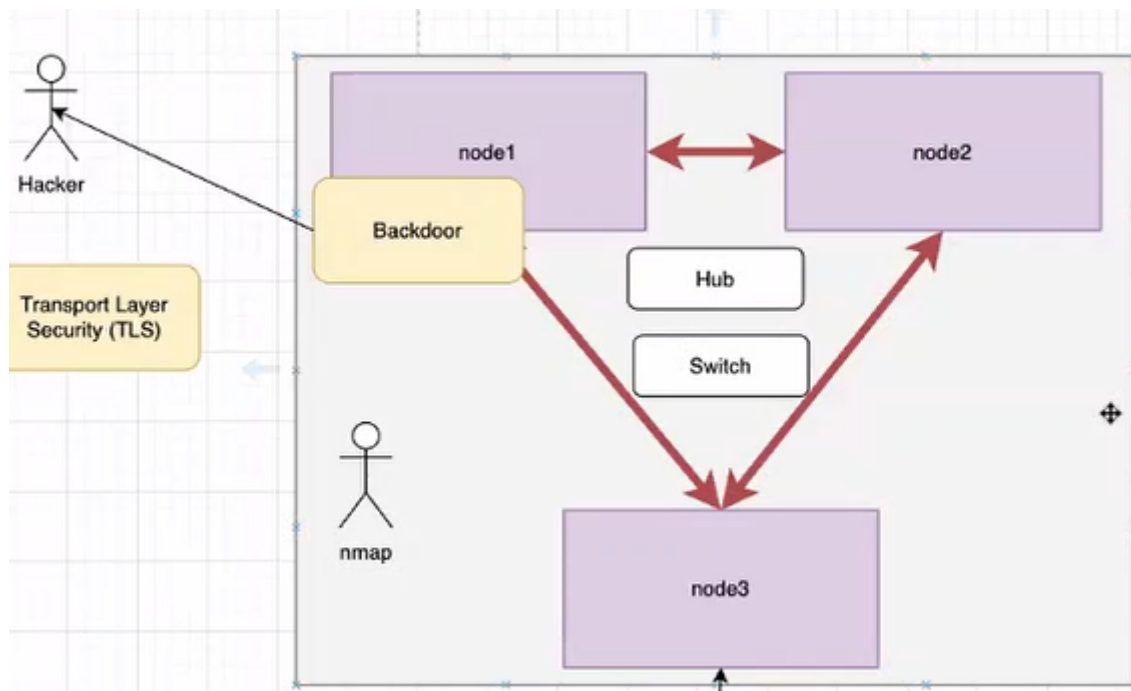
Cuando hay más de 30 conexiones por segundo que no parecen legítimas, se asocia a un ataque DoS. Lo que se está intentando es generar tantos threads y context switches que el servicio se empeore o deje de funcionar.



El DDoS consiste en tener un montón de ubicaciones alrededor del mundo enviando peticiones que parecen válidas. Este tipo de ataque se puede aplicar a un sitio web, a una API o incluso a una base de datos, cualquiera de ellos puede colapsar si recibe mucha carga. La idea es generar tanto ruido y tantas peticiones simultáneas que el servidor continúe creyendo que todo es normal, pero en realidad está siendo saturado hasta el punto en que el servicio deja de funcionar correctamente.

Backdoor

Una backdoor ocurre cuando alguien logra intervenir el servidor de la base de datos y establece una conexión que envía información a otra persona que está robando esos datos. Para lograrlo, pueden usar herramientas como nmap o aprovechar cualquier vulnerabilidad que exista en la red. Este tipo de ataque no solo puede hacerse en servidores también pueden instalar una backdoor en nuestras computadoras o en nuestros teléfonos.



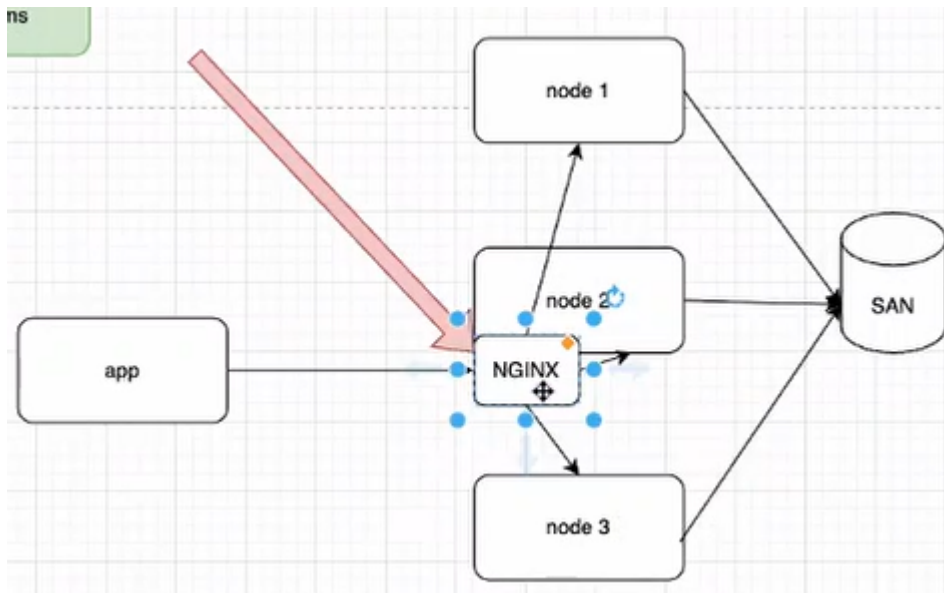
Packet Sniffing

Está relacionado con el ataque de man in the middle. Consiste en capturar paquetes que se están moviendo por la red. Si esos paquetes no están encriptados, simplemente se puede robar la información que llevan.

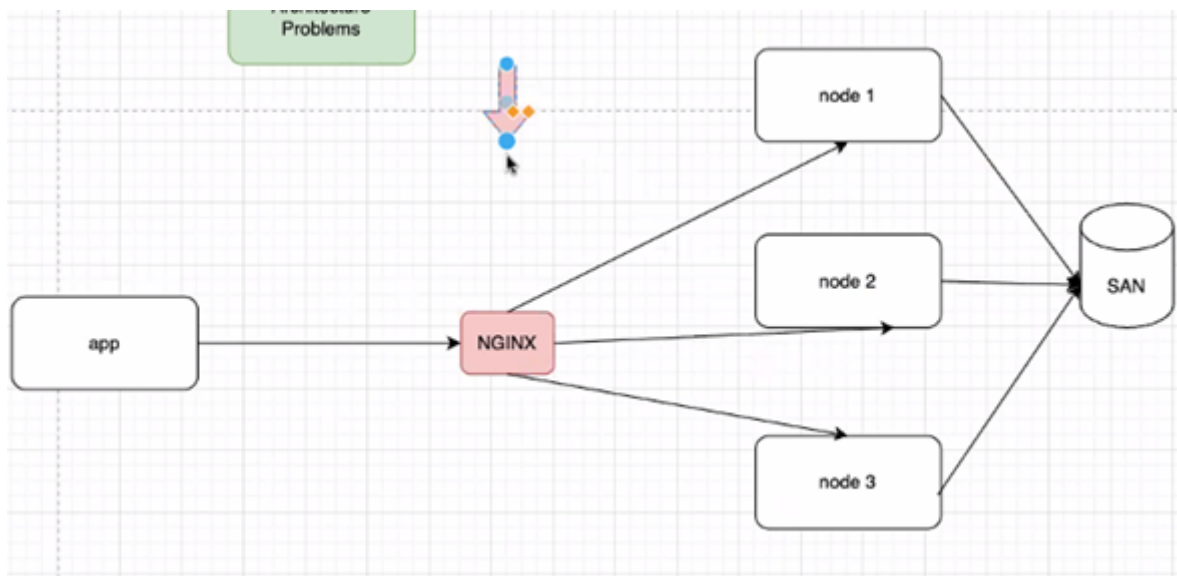
Architecture Problems

Suponga que se realiza la instalación de un servidor o de 3 nodos. Una vez que se tienen esos 3 nodos, el storage utiliza un SAN. Luego, alguien decide que quiere balancear la carga entre esos 3 servidores utilizando un software llamado NGINX, colocándolo en el nodo 2. Con esto, NGINX se encarga de balancear la carga entre todos los nodos, y cuando la aplicación quiere entrar a la base de datos, lo hace a través del NGINX.

¿Dónde está el problema de arquitectura? Al poner NGINX como punto central, se crea un single point of failure. Si NGINX no es lo suficientemente estable, al caerse NGINX también se cae el acceso a la base de datos. En otras palabras, si el NGINX falla, se mata la conexión hacia la base de datos.

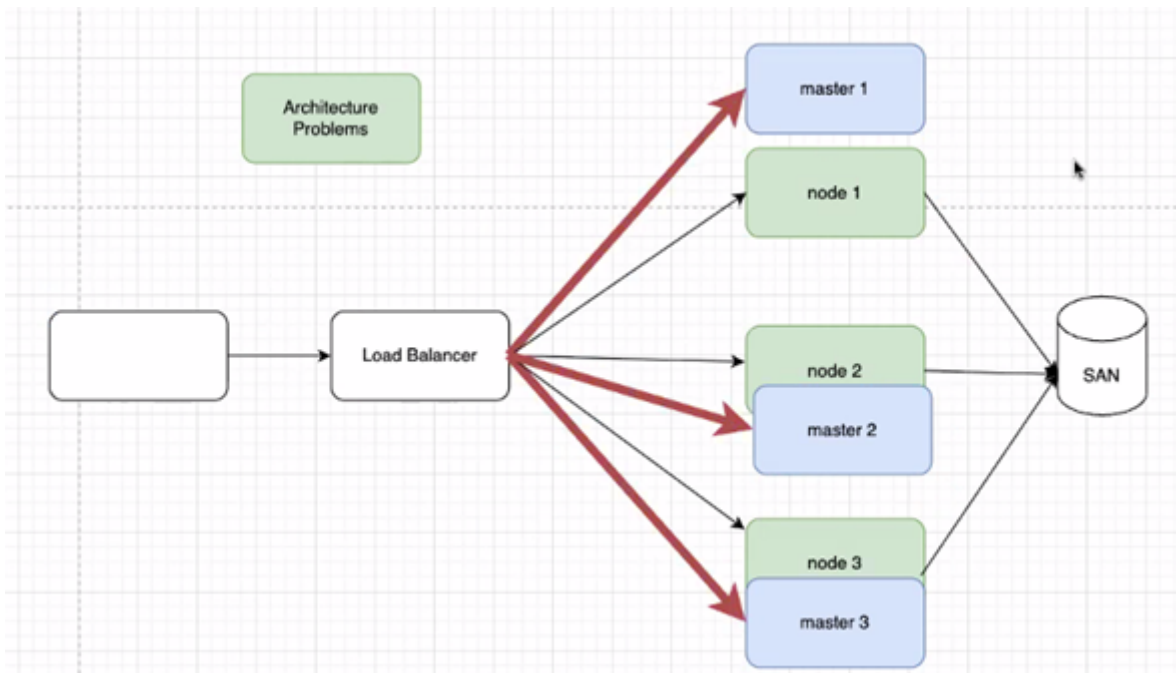


Pero, ¿qué pasa si pongo el NGINX aparte o fuera de esos nodos? Sigo teniendo un problema si no le configuro correctamente la memoria y el CPU, porque no podría manejar toda la carga que se esté generando. Además, aunque esté separado, si el NGINX se cae, igual deja de servirme y se afecta el acceso a los demás servicios.



Otro problema

Suponga ahora que se tiene otro problema de arquitectura. Tenemos un clúster de Elasticsearch y se hace todo muy bien a nivel de hardware y load balancing. Los servidores manejan toda la carga de escritura del sistema sin problema. Si alguien quiere bajar el sistema y lo ataca, el problema de arquitectura es que los master nodes no deberían participar en el load balancer, porque no tienen la misma capacidad que los data nodes ni manejan la misma cantidad de consultas simultáneas. Si los master nodes empiezan a recibir demasiada carga, el sistema se va a poner tan lento que ni siquiera va a poder atender las funciones básicas que les corresponden.



Weak Passwords y Password Cracking

Estos dos conceptos están muy relacionados. Si alguien se mete al dark web, puede encontrar bases de datos completas de contraseñas. El uso de contraseñas débiles debería estar totalmente erradicado del día a día, porque incluso se pueden usar herramientas De inteligencia artificial para rastrear y predecir contraseñas. Las contraseñas débiles exponen los sistemas, y conforme la IA avanza, es más fácil hackearlas.

En cuanto al password cracking, normalmente se usa fuerza bruta para guiar la búsqueda de la contraseña. Si se sabe que una base de datos soporta contraseñas basadas en los caracteres visibles de la tabla ASCII, el atacante empieza a probar permutaciones hasta encontrar la contraseña real almacenada en la base. Por eso no se deben usar contraseñas débiles.

Causas de fallo en bases de datos

Power Failure

En cualquier momento se puede ir la luz. Si tengo todos mis nodos en una misma ubicación, se pueden perder los datos completos, incluso toda la base de datos. Si los nodos están en el mismo país también existe riesgo. En Costa Rica no es tan común, pero si todo está con la misma compañía eléctrica es posible que ocurra.

Hardware Failure

Siempre existe la posibilidad de que un servidor o una parte del hardware falle. Por eso siempre se debe tener redundancia para evitar perder servicios o datos.

Transaction Failure

Puede ocurrir cuando hay un deadlock y una transacción no puede continuar.

Software Corruption

La mayor causa de corrupción en una base de datos es cuando se hace una actualización de un componente y no se realiza correctamente. Si por error se introduce un breaking change al actualizar, el sistema queda corrupto y la base de datos se cae. Cuando tenemos la infraestructura in-house, es mucho más difícil recuperarse ante este tipo de fallos que cuando está en la nube.

Evento catastrófico

Pueden suceder en cualquier momento y está fuera de nuestro control, hay que tomar buenas medidas.

RPO

Es el tiempo máximo que transcurre desde el último momento de recuperación de datos y determina si la pérdida de datos es aceptable o no dependiendo del punto de recuperación e interrupción de la bd. Si define la condición de que solo por 15 minutos pueden perderse datos, eso debe cumplirse y no debe ser más.

RTO

(Tiempo de Recuperación Total) es el tiempo máximo aceptable entre la interrupción de la base de datos y la restauración del servicio. Ayuda a determinar el tiempo aceptable en el cual la bd no estuvo disponible. Por ejemplo si se pone la condición de que el tiempo máximo de recuperación es de 5 minutos, debe cumplirse que todo esté operando de nuevo en este tiempo.

Business Continuity Plan

Mantener las funciones esenciales durante y después de una interrupción natural o causada por el ser humano. Cuando un desastre provoca que las operaciones se detengan por completo, se deben tomar medidas para que la organización pueda volver a la normalidad lo antes posible.

Disaster Recovery Plan

Restaurar las operaciones críticas del negocio después de un desastre.